

# UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación  
Carrera de Ingeniería en Software

---

## Herramientas Automatizadas para la Ingeniería de Requisitos:

rmToo y el Paradigma Requirements as Code  
Informe técnico

## Asignatura:

Ingeniería de Requisitos

## Docente:

Ing. Gleiston Cicerón Guerrero Ulloa

## Curso / Paralelo:

4.º Software “A”

## Grupo H:

- Huilcapi Leon Denisses Fabiola
- Tigasi Sampedro Paul Alexander
- Vaca Romero David Octavio

## Repositorio:

[HuilcapiD/HTV\\_GRUPO\\_H/SEMANA\\_6 at main · Fabi06-ux/HuilcapiD](#)

## Contenido

|                                                                  |    |
|------------------------------------------------------------------|----|
| 1. Introducción.....                                             | 3  |
| 2. Herramienta 1 — rmToo: el paradigma Requirements as Code..... | 3  |
| 3. Herramienta 2 — IBM DOORS Next.....                           | 5  |
| 4. Comparativa de funcionalidades.....                           | 5  |
| 5. Ventajas y limitaciones.....                                  | 6  |
| 6. Aplicabilidad al Proyecto Fin de Curso (PFC).....             | 7  |
| 7. Manual de usuario.....                                        | 8  |
| 8. Conclusión.....                                               | 9  |
| Referencias.....                                                 | 10 |

# 1. Introducción

---

La ingeniería de requisitos (IR) es el proceso sistemático de elicitación, especificación, validación y gestión de los requisitos de un sistema [11], [13]. Durante décadas esta disciplina ha dependido de archivos de Word, PDF y wikis como medio principal para registrar lo que un sistema debe hacer. Este enfoque tiene un problema estructural: mezcla el contenido con el formato, lo que dificulta el procesamiento automático, el versionado y la trazabilidad de los requisitos.

El estándar ISO/IEC/IEEE 29148:2018 establece que un requisito de calidad debe ser, entre otros atributos, verificable, no ambiguo, completo y trazable [12]. Alcanzar estas propiedades con documentación narrativa no estructurada es difícil, precisamente porque el documento no es procesable por máquina. El cuerpo de conocimiento SWEBOK v4 reconoce la gestión y la trazabilidad de requisitos como actividades centrales del área de IR [13].

Una revisión sistemática de literatura publicada en la revista *Requirements Engineering* de Springer [1], que analizó investigaciones sobre automatización en IR desde 1996 hasta 2022, concluye que ejecutar las actividades de requisitos de forma manual es costoso en tiempo y propenso a errores; la automatización reduce los errores de inconsistencia y el esfuerzo de actualización. De forma complementaria, un estudio publicado en *Information* [2], que revisa artículos de Scopus e IEEE Xplore sobre retos de la IR ágil, señala que la falta de trazabilidad formal y la dependencia de documentación no estructurada son dos de los problemas más frecuentes reportados por los equipos de desarrollo.

Este informe analiza y compara dos herramientas automatizadas de IR de filosofías opuestas: *rmToo*, de código abierto y orientada a texto plano (*Requirements as Code*), e *IBM Engineering Requirements Management DOORS Next*, comercial y centrada en una interfaz gráfica colaborativa. A partir de la comparación se discuten sus ventajas, limitaciones y su aplicabilidad al Proyecto Fin de Curso (PFC), y se incluye un manual de instalación y uso de *rmToo*.

## 2. Herramienta 1 — *rmToo*: el paradigma *Requirements as Code*

---

El *rmToo* es una herramienta de gestión de requisitos libre y de código abierto (licencia GPL v3), escrita en Python, que procesa los requisitos a partir de archivos de texto plano y genera automáticamente documentación HTML, documentos LaTeX/PDF, grafos de dependencias y reportes estadísticos [7]. Es un proyecto activo, mantenido desde 2010 con versiones recientes en 2025–2026 [7].

### 2.1. El paradigma *Requirements as Code*

El concepto que sustenta *rmToo* es tratar los requisitos con la misma rigurosidad con que el desarrollo moderno trata el código fuente: versionados, procesables automáticamente, auditables y separados de su presentación visual. Este principio se conoce como *Requirements as Code* y se alinea con la noción de requisito gestionable y trazable descrita por Pohl [11].

### 2.2. Separación de contenido y formato

Al igual que Donald Knuth separó contenido y presentación con TeX en 1984, *rmToo* separa la definición de un requisito —escrita en un archivo de texto plano— de su renderizado final como PDF vía LaTeX, grafos de trazabilidad vía Graphviz o documentación web en HTML. Los mismos archivos pueden producir distintos formatos de salida sin modificar el contenido.

### 2.3. Texto plano y control de versiones con Git

Al almacenarse en texto plano, los archivos de requisitos son completamente compatibles con Git. Esto permite aplicar sobre los requisitos las mismas prácticas que el equipo ya aplica al código:

- Historial de cambios completo (quién cambió qué, cuándo y por qué).
- Ramas para trabajar cambios en paralelo.
- Pull requests para revisión por pares.
- Reversión de errores a estados anteriores.

El proyecto RTEMS —un sistema operativo de tiempo real de código abierto con requisitos de trazabilidad estrictos— evaluó múltiples herramientas y seleccionó rmToo y Doorstop como las únicas candidatas viables, precisamente por su capacidad de integrarse con Git sin bases de datos ni entornos propietarios [8].

### 2.4. Trazabilidad automatizada y su impacto en la calidad

La trazabilidad de requisitos es la capacidad de establecer y mantener vínculos entre los requisitos y los demás artefactos del proyecto —diseño, código, pruebas y documentación— y es fundamental para el análisis de impacto y la verificación de completitud [11], [12]. Un estudio publicado en PeerJ Computer Science [3] demuestra que la creación y el mantenimiento manual de matrices de trazabilidad es tan costoso que, en la mayoría de los proyectos, directamente no se realiza; los autores muestran que, cuando la trazabilidad se genera automáticamente desde los artefactos, su adopción aumenta de forma significativa.

Un trabajo complementario en Requirements Engineering [4] identificó las dos principales barreras para implementar trazabilidad: el esfuerzo percibido como excesivo y la falta de integración con las herramientas existentes. rmToo elimina ambas: la trazabilidad se genera automáticamente desde los archivos .req y las herramientas necesarias (Git y terminal) son las que el equipo ya usa. Además, la revisión sistemática [1] reporta que, cuando los requisitos se tratan como artefactos estructurados y trazables, los equipos experimentan menos ambigüedad y, en consecuencia, menos defectos durante la implementación; esto es coherente con el grafo de dependencias que rmToo genera, que permite detectar ciclos o dependencias rotas antes de llegar al código.

### 2.5. Integración con CI/CD y Living Documentation

Una de las capacidades más relevantes de rmToo es su integración natural con flujos de integración y despliegue continuo (CI/CD), lo que la distingue de las soluciones centradas en GUI que deben actualizarse manualmente. El concepto de Living Documentation describe documentación técnica que se genera automáticamente a partir del código y los artefactos del proyecto, de modo que siempre refleja el estado actual del sistema. Un capítulo de Springer de 2023 [5] describe cómo los pipelines CI/CD pueden automatizar la generación de documentación técnica en proyectos SaaS, eliminando la desincronización entre el sistema y su documentación.

Esto es exactamente lo que rmToo hace con los requisitos: cada commit que modifica archivos .req puede disparar la regeneración automática del PDF, del grafo de trazabilidad y de la documentación web. Un estudio de 2022 presentado en IEEE ICSME [6] analizó miles de repositorios y encontró que la adopción de GitHub Actions ha crecido sostenidamente desde 2019, siendo la automatización de documentación uno de los casos de uso más frecuentes; al basarse en texto plano y Python, rmToo puede ejecutarse desde cualquier Action sin configuraciones especiales.

## 3. Herramienta 2 — IBM Engineering Requirements Management DOORS Next

IBM DOORS (Dynamic Object-Oriented Requirements System) es una de las herramientas de gestión de requisitos más consolidadas del mercado, especialmente en dominios regulados como aeroespacial, defensa, dispositivos médicos y automoción. Surgió en 1991 y hoy existe en dos productos arquitectónicamente distintos: DOORS Classic (versión 9.x), el cliente de escritorio original, y DOORS Next, una aplicación web construida sobre la plataforma abierta Jazz [14].

### 3.1. Arquitectura e interoperabilidad

La arquitectura de DOORS Next se basa en la especificación OSLC (Open Services for Lifecycle Collaboration), que utiliza recursos y servicios REST comunes para compartir datos entre aplicaciones del ciclo de vida. El intercambio de requisitos con otras herramientas se realiza mediante ReqIF (Requirements Interchange Format), estándar gobernado por el Object Management Group (OMG), lo que permite que equipos de distintas organizaciones trabajen sobre especificaciones compartidas [14].

### 3.2. Trazabilidad y gestión de cambios

La trazabilidad es el núcleo de DOORS Next: ofrece enlaces tipados definidos por el sistema (por ejemplo Satisfies, Parent/Child, Derives) y enlaces personalizados, con trazabilidad multinivel a través de módulos. Cuando un requisito enlazado cambia, los vínculos afectados se marcan como «sospechosos» (suspect links), de modo que el equipo identifica rápidamente qué artefactos requieren revisión [14]. Para la gestión de cambios, la herramienta captura líneas base (baselines) que fijan una instantánea del proyecto, conserva el historial completo de revisiones de cada artefacto y permite restaurar estados anteriores, además de gestionar configuraciones globales entre disciplinas [14].

### 3.3. Colaboración e inteligencia artificial

Al ser una aplicación web multiusuario, DOORS Next provee colaboración en tiempo real, paneles (dashboards), reportes y vistas altamente personalizables sobre atributos como prioridad, estado o responsable [14]. Las versiones recientes de la familia DOORS incorporan además capacidades asistidas por inteligencia artificial orientadas a evaluar la calidad de los requisitos y automatizar tareas de cumplimiento en entornos regulados [14].

## 4. Comparativa de funcionalidades

El mercado de herramientas de gestión de requisitos se divide en dos grandes familias: las herramientas centradas en interfaz gráfica (GUI) y las orientadas a texto plano y código. La Tabla 1 contrasta cinco herramientas representativas según seis criterios pertinentes para la IR moderna.

| Herramienta    | Trazabilidad                               | Gestión de cambios                    | Colaboración                         | IA                           | Costo                     | Facilidad de uso                  |
|----------------|--------------------------------------------|---------------------------------------|--------------------------------------|------------------------------|---------------------------|-----------------------------------|
| rmToo          | Automática: grafo Graphviz desde .req      | Git (commits, ramas, pull requests)   | Asíncrona vía Git/PR                 | No nativa                    | Gratis (GPL v3)           | Baja (CLI; curva alta)            |
| IBM DOORS Next | Multinivel; enlaces tipados; suspect links | Baselines + historial; config. global | Alta (web, multiusuario, dashboards) | Sí (calidad asistida por IA) | Alto (licencia comercial) | Media (GUI rica; admin. compleja) |

| Herramienta               | Trazabilidad                         | Gestión de cambios               | Colaboración                        | IA                               | Costo              | Facilidad de uso |
|---------------------------|--------------------------------------|----------------------------------|-------------------------------------|----------------------------------|--------------------|------------------|
| <b>Jama Connect</b>       | Matriz en vivo; cobertura de pruebas | Versionado + revisiones formales | Alta (SaaS; revisiones en línea)    | Sí (asistente generativo)        | Alto (suscripción) | Media-alta       |
| <b>Doorstop</b>           | Enlaces en YAML; validación          | Git nativo                       | Asíncrona vía Git                   | No                               | Gratis (LGPL v3)   | Baja-media (CLI) |
| <b>Word / Google Docs</b> | Manual; no estructural               | Control de cambios básico        | Media (edición simultánea en GDocs) | Parcial (asistentes generativos) | Gratis / comercial | Alta             |

**Tabla 1.** Comparación de herramientas de gestión de requisitos según seis criterios.

#### 4.1. Análisis interpretativo

Las dos familias optimizan objetivos distintos. Las herramientas orientadas a código (rmToo, Doorstop) maximizan la automatización, el versionado y la integración con CI/CD a costo cero, pero sacrifican la facilidad de uso y la colaboración en tiempo real, ya que carecen de GUI. Las herramientas comerciales (DOORS Next, Jama Connect) ofrecen colaboración rica, gobernanza, trazabilidad multinivel y, cada vez más, asistencia por IA, a cambio de un costo de licencia elevado y una administración compleja. Word y Google Docs, aunque accesibles, no proporcionan trazabilidad estructural y su gestión de cambios es limitada.

Esta lectura es coherente con la evidencia: un estudio sistemático de Springer [9] concluyó que las herramientas basadas en texto plano con integración Git ofrecen ventajas claras en contextos de desarrollo ágil, y un trabajo en IEEE Access [10] subraya que los equipos con prácticas establecidas de CI/CD se benefician más de herramientas estructuradas y procesables que de documentos narrativos.

Conclusión fundamentada: para un equipo académico o ágil pequeño que ya utiliza Git y CI/CD —como el del PFC—, una herramienta como rmToo aporta trazabilidad automatizada sin costo y con baja fricción de adopción [1], [9], [10]. En cambio, una organización grande con exigencias regulatorias justifica el costo de DOORS Next o Jama Connect por su gobernanza, colaboración y capacidades de IA.

## 5. Ventajas y limitaciones

### 5.1. rmToo

#### Ventajas

- **Trazabilidad automatizada:** el grafo de dependencias se genera desde los req, eliminando el costo del mantenimiento manual de matrices [3], [4].
- **Integración con Git y CI/CD:** versionado completo y regeneración automática de artefactos en cada commit [6].
- **Costo nulo y código abierto:** licencia GPL v3, sin dependencia de proveedores [7].
- **Validación en la práctica:** seleccionada por el proyecto RTEMS por su trazabilidad y compatibilidad con Git [8].

#### Limitaciones

- **Ausencia de interfaz gráfica:** está pensada para programadores y se opera por línea de comandos, lo que eleva la curva de aprendizaje [7].
- **Dependencias externas:** requiere LaTeX para PDF y Graphviz para los grafos, lo que complica la instalación inicial [7].

- **Colaboración asíncrona:** no ofrece edición simultánea ni paneles en línea; la coordinación depende del flujo de Git.

## 5.2. IBM DOORS Next

### Ventajas

- **Gobernanza y trazabilidad multinivel:** enlaces tipados, suspect links y líneas base para entornos regulados [14].
- **Interoperabilidad estándar:** integración mediante OSLC y ReqIF con otras herramientas del ciclo de vida [14].
- **Colaboración y reportes:** aplicación web multiusuario con dashboards y vistas personalizables [14].
- **Capacidades de IA:** evaluación de calidad de requisitos asistida por IA en las versiones recientes [14].

### Limitaciones

- **Costo elevado:** licenciamiento comercial que dificulta su adopción en contextos académicos [14].
- **Complejidad de administración:** su configuración y mantenimiento exigen personal especializado [14].
- **Migración no trivial:** DOORS Classic y DOORS Next son arquitecturas separadas; pasar de una a otra es una migración, no una actualización [14].

## 6. Aplicabilidad al Proyecto Fin de Curso (PFC)

Esta sección vincula funcionalidades concretas de rmToo con artefactos y requisitos reales del Proyecto Fin de Curso del equipo: el Sistema Inteligente de Mantenimiento de Palma Africana (SIMPA), un sistema que analiza imágenes de las palmas mediante inteligencia artificial para detectar plagas, enfermedades y deficiencias nutricionales, y cuya Especificación de Requisitos de Software (ERS) sigue la norma ISO/IEC/IEEE 29148:2018 [12]. La Tabla 2 mapea cada actividad del proceso de IR [11], [13] con un requisito o artefacto real del SIMPA, la funcionalidad de rmToo que lo soporta y la evidencia que esta genera.

| Actividad del proceso de IR         | Artefacto / requisito del PFC                                                                                                                            | Funcionalidad de rmToo                                                           | Evidencia generada                                    |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------|
| <b>Elicitación y especificación</b> | RF-07 «Detección de enfermedades» ( $\geq 10$ enfermedades) y RF-08 «Detección de plagas» ( $\geq 5$ plagas), del módulo de análisis de imágenes con IA. | Cada RF/RNF se define en un archivo .req en texto plano.                         | ERS en HTML/PDF generada automáticamente.             |
| <b>Modelado de dependencias</b>     | Cadena RF-04 «Captura de imágenes» $\rightarrow$ RF-06 «Análisis automático» $\rightarrow$ RF-07 «Detección de enfermedades».                            | Campo Depends on en los .req; grafo con Graphviz.                                | Grafo de trazabilidad de la cadena de análisis.       |
| <b>Verificación y validación</b>    | RNF-03 «Precisión del modelo IA $\geq 90\%$ » y RNF-05 «Cifrado de contraseñas $\geq 256$ bits», verificables (ISO/IEC/IEEE 29148).                      | Detección de ciclos/dependencias rotas; campo Status para auditar la validación. | Reporte de inconsistencias y de estado por requisito. |
| <b>Gestión y control de cambios</b> | Cambios sobre el backlog del SIMPA versionados en el repositorio GitHub del equipo (Villafuerte_Grupo_AHMRV).                                            | Versionado con Git: commits, ramas y pull requests sobre los .req.               | Historial de cambios + ERS regenerada en cada commit. |
| <b>Priorización / planificación</b> | Requisitos del primer sprint: RF-01 «Registro de usuarios», RF-02 «Inicio de sesión» y RF-03 «Registro de plantaciones».                                 | Campos Priority y Effort estimation en los .req.                                 | Backlog priorizado y lista de prioridades (LaTeX).    |

**Tabla 2.** Mapeo de funcionalidades de rmToo a las actividades de IR del SIMPA (PFC del equipo).



En síntesis, el SIMPA se beneficia de rmToo porque convierte sus requisitos —los RF de análisis de imágenes y los RNF de precisión, seguridad y rendimiento— en artefactos versionados y trazables: cada requisito queda enlazado a sus dependencias, su estado de validación es auditable y cualquier cambio en el backlog, versionado en el repositorio GitHub del equipo, regenera automáticamente la ERS y el grafo de trazabilidad, reduciendo el riesgo de inconsistencias durante el desarrollo [1], [3].

## 7. Manual de usuario

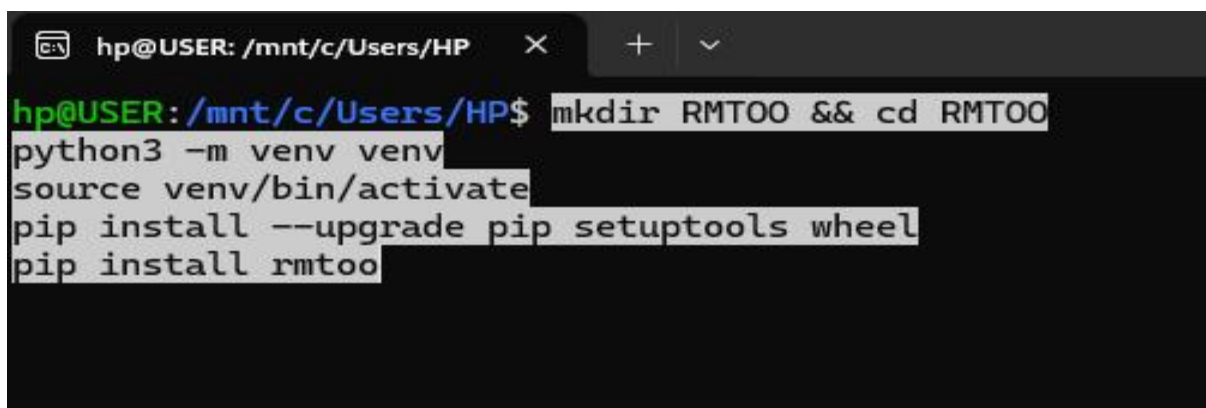
Esta sección documenta la instalación y el uso básico de rmToo aplicado a los requisitos del SIMPA. Las figuras deben reemplazarse por capturas propias del equipo en las que se observen los requisitos reales del proyecto.

### 7.1. Requisitos previos

- Python 3 (rmToo está escrito en Python).
- LaTeX, necesario para generar los documentos PDF.
- Graphviz, necesario para generar el grafo de dependencias.

### 7.2. Instalación

Se recomienda instalar rmToo dentro de un entorno virtual. En una terminal Linux:

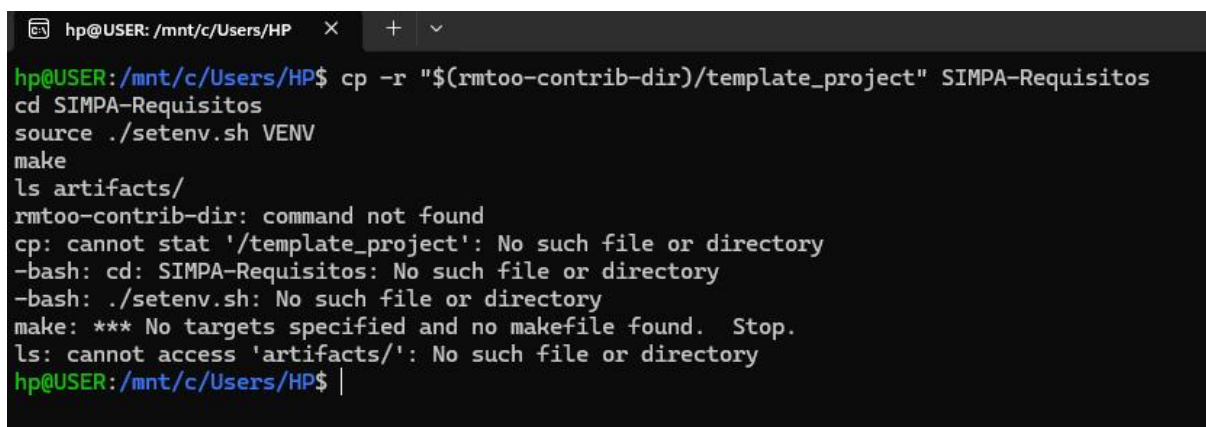


```
hp@USER: /mnt/c/Users/HP$ mkdir RMT00 && cd RMT00
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip setuptools wheel
pip install rmt00
```

Figura 1. Instalación de rmToo finalizada en la terminal del equipo (salida de `pip install rmt00`).

### 7.3. Creación del proyecto

rmToo incluye un proyecto plantilla. Se copia, se cargan las variables de entorno y se genera por primera vez:



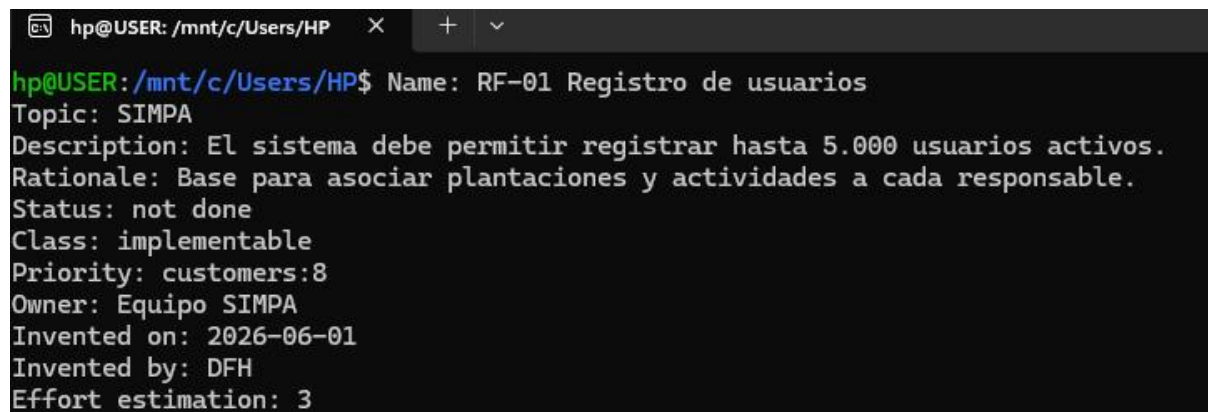
```
hp@USER: /mnt/c/Users/HP$ cp -r "$(rmt00-contrib-dir)/template_project" SIMPA-Requisitos
cd SIMPA-Requisitos
source ./setenv.sh VENV
make
ls artifacts/
rmt00-contrib-dir: command not found
cp: cannot stat '/template_project': No such file or directory
-bash: cd: SIMPA-Requisitos: No such file or directory
-bash: ./setenv.sh: No such file or directory
make: *** No targets specified and no makefile found. Stop.
ls: cannot access 'artifacts/': No such file or directory
hp@USER: /mnt/c/Users/HP$
```

Figura 2. Estructura del proyecto del SIMPA creada a partir de la plantilla de rmToo.



## 7.4. Definición de los requisitos del PFC

Cada requisito se define en un archivo .req dentro de la carpeta de requisitos. A modo de ejemplo, el requisito RF-01 del SIMPA se especifica así:

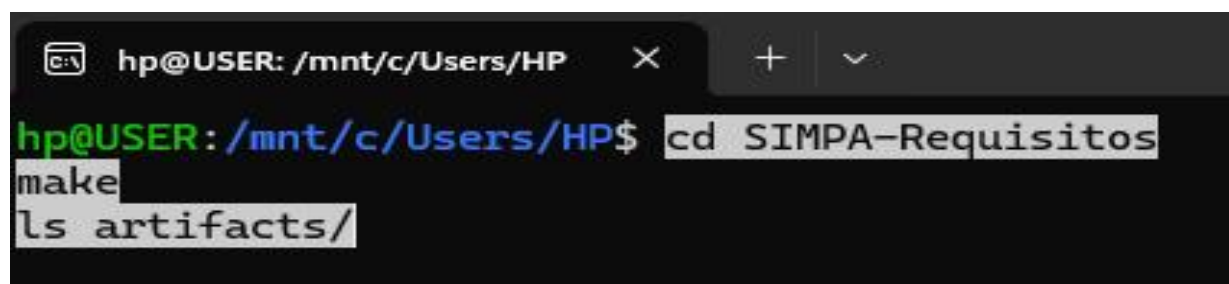


```
hp@USER: /mnt/c/Users/HP$ Name: RF-01 Registro de usuarios
Topic: SIMPA
Description: El sistema debe permitir registrar hasta 5.000 usuarios activos.
Rationale: Base para asociar plantaciones y actividades a cada responsable.
Status: not done
Class: implementable
Priority: customers:8
Owner: Equipo SIMPA
Invented on: 2026-06-01
Invented by: DFH
Effort estimation: 3
```

Figura 3. Requisito real del SIMPA (RF-01) definido en un archivo .req.

## 7.5. Generación de artefactos

Tras añadir o modificar requisitos, se ejecuta nuevamente make; rmToo regenera automáticamente la especificación (HTML/PDF), el grafo de trazabilidad y los reportes estadísticos en la carpeta artifacts/.



```
hp@USER: /mnt/c/Users/HP$ cd SIMPA-Requisitos
make
ls artifacts/
```

Figura 4. Especificación en PDF..

## 8. Conclusión

La evidencia académica revisada confirma que el enfoque de rmToo responde directamente a problemas documentados en la literatura: la automatización en IR reduce errores [1], la trazabilidad mejora la calidad del software [3], [4] y la integración con CI/CD elimina la desincronización entre requisitos y sistema [5], [6]. Frente a una herramienta comercial como IBM DOORS Next superior en colaboración, gobernanza y capacidades de IA, pero costosa y compleja rmToo ofrece a un equipo técnico que ya trabaja con Git una solución de trazabilidad automatizada sin costo y de baja fricción [9], [10].

rmToo tiene limitaciones reales ausencia de interfaz gráfica, dependencias de LaTeX y Graphviz y curva de aprendizaje inicial; sin embargo, estas limitaciones se justifican para el contexto del PFC, donde el equipo ya domina Git y la línea de comandos y prioriza la trazabilidad automatizada y la documentación viva por encima de la facilidad de uso.

## Referencias

---

- [1] K. Lano et al., “Advances in automated support for requirements engineering: a systematic literature review,” *Requirements Engineering*, vol. 29, no. 2, pp. 177–207, Jun. 2024, doi: 10.1007/s00766-023-00411-0.
- [2] Z. Hoy and M. Xu, “Agile software requirements engineering challenges-solutions: A conceptual framework from systematic literature review,” *Information*, vol. 14, no. 6, p. 322, Jun. 2023, doi: 10.3390/info14060322.
- [3] M. J. Escalona, N. Koch, and L. Garcia-Borgoñon, “Lean requirements traceability automation enabled by model-driven engineering,” *PeerJ Comput. Sci.*, vol. 8, e817, Jan. 2022, doi: 10.7717/peerj-cs.817.
- [4] M. Ruiz, J. Y. Hu, and F. Dalpiaz, “Why don’t we trace? A study on the barriers to software traceability in practice,” *Requirements Engineering*, vol. 28, pp. 619–637, 2023, doi: 10.1007/s00766-023-00408-9.
- [5] B. S. Satapathy, S. S. Satapathy, S. I. Singh, and J. Chakraborty, “Continuous Integration and Continuous Deployment (CI/CD) Pipeline for the SaaS Documentation Delivery,” in *Lecture Notes in Electrical Engineering*, vol. 1080. Singapore: Springer, 2023, doi: 10.1007/978-981-99-5994-5\_5.
- [6] A. Decan, T. Mens, P. Rostami Mazrae, and M. Golzadeh, “On the use of GitHub Actions in software development repositories,” in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2022, pp. 235–245, doi: 10.1109/ICSME55016.2022.00029.
- [7] A. Florath, *rmToo: Free and Open Source Requirements Management Tool [Software]*, versions 2010–2026, flonatel GmbH & Co. KG. [Online]. Available: <https://github.com/florath/rmtoo>
- [8] RTEMS Project, “Tooling — RTEMS Software Engineering Documentation,” RTEMS Docs, Jan. 2026. [Online]. Available: <https://docs.rtems.org/docs/main/eng/req/tooling.html>
- [9] M. Kamalrudin et al., “Evaluation of Open Source Tools for Requirements Management,” in *Proc. Int. Conf. Product-Focused Software Process Improvement (PROFES)*, Springer, 2020, pp. 175–191, doi: 10.1007/978-3-030-48325-8\_13.
- [10] W. Behutiye, P. Rodríguez, and M. Oivo, “Quality Requirement Documentation Guidelines for Agile Software Development,” *IEEE Access*, vol. 10, pp. 70154–70173, 2022, doi: 10.1109/ACCESS.2022.3187456.
- [11] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Berlin, Heidelberg: Springer, 2010.
- [12] ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*. Geneva, Switzerland: ISO/IEC/IEEE, 2018.
- [13] IEEE Computer Society, *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*. Piscataway, NJ, USA: IEEE, 2024.
- [14] IBM Corporation, “Overview of IBM Engineering Requirements Management DOORS Next,” *IBM Documentation (ELM Suite, DOORS Next 7.2.0)*, 2024. [Online]. Available: <https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/doors-next>

# El Poder del Texto Plano

*“rmToo Best for text-based requirements”*

## INTEGRANTES / GRUPO H:

- HUILCAPI LEON DENISSES FABIOLA
- TIGASI SAMPEDRO PAUL ALEXANDER
- VACA ROMERO DAVID OCTAVIO

Automatización Avanzada en la  
Ingeniería de Requisitos con rmToo



# ¿Y si tratáramos los requerimientos como código?

## ✗ MODELO TRADICIONAL

- Documento Word / PDF / Google Docs
- Contenido y formato mezclados
- Sin historial de cambios real
- Imposible procesar automáticamente
- Si se pierde, se pierde todo



## ✓ REQUIREMENTS AS CODE



- Archivos de texto plano (.req)
- Contenido separado del formato
- Git: historial completo y auditable
- Motor rmToo procesa y compila
- Portable, libre, soberano

# Pipeline de rmToo — Cómo Funciona

01

## Entrada

Archivos .req  
en texto plano  
puro por el  
ingeniero



02

## Motor rmToo

Python analiza  
dependencias,  
valida reglas  
y construye grafo



03

## Salida

PDF (LaTeX)  
Grafo (Graphviz)  
HTML  
Siempre actualizados



# rmToo vs Herramientas del Mercado

| Herramienta   | Control Git | Costo    | Trazabilidad | Datos          |
|---------------|-------------|----------|--------------|----------------|
| IBM DOORS     | ✗ No nativo | \$\$\$\$ | ✓ Sí         | 🔒 Propietario  |
| Jama Software | ✗ No nativo | \$\$\$   | ✓ Sí         | 🔒 Propietario  |
| ChatGPT / AI  | ✗ No        | Gratis   | ✗ No         | ☁️📁 Nube ajena |
| rmToo         | ✓ Total     | Gratis   | ✓ Nativo     | ✓ Tus archivos |

rmToo sacrifica la GUI para ganar control total, ligereza y soberanía de datos.

# Ventajas y Limitaciones

## ✓ VENTAJAS

### Ligereza Extrema

Open Source · pip install · sin servidores ni licencias



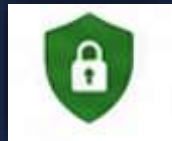
### CI/CD Nativo

GitHub Actions genera docs automáticamente en cada commit



### Datos Soberanos

Sus .req viven en su repo · nadie puede quitárselos



## ⚠ LIMITACIONES

### Sin GUI

Todo en terminal y editor de texto · requiere perfil técnico



### Curva de Aprendizaje

Necesita fluidez en Git, CLI y sintaxis .req



### Fricción con Stakeholders

No técnicos necesitan capa intermedia para editar





# Respaldo Científico y Tendencias

2024



Springer — Requirements Engineering Journal

Requerimientos estructurados logran mayor consistencia, menos ambigüedad y ciclos de revisión más cortos.

2023



IEEE — RE'23 Conference

Tendencia hacia Living Documentation: docs que se autogeneran y siempre reflejan el estado actual.

2022



IEEE — Estudio de Trazabilidad

Trazabilidad completa reduce significativamente los defectos en proyectos de tamaño mediano a grande.

rmTop implementa estas 3 conclusiones de forma nativa, sin herramientas adicionales.

# Demo Práctica — Sintaxis .req

```
Configurar rmToo para di X + - □ X
Write(requirements/sgu-perfil.req)
Write(requirements/sgu-baja.req)
Write(requirements/sgu-log.req)

Overwrite file
requirements/sgu-auth.req

3 Invented on: 2026-06-15
4 Invented by: flonatel
5 Owner: users
6 Description: El sistema debe \textbf{autenticar} a los usuarios registrados validando su correo electrónico y con-
  -traseña antes de conceder acceso.
7 Rationale: La autenticación es la primera línea de defensa contra accesos no autorizados.
6 Description: Autentica usuarios validando correo y contraseña antes de dar acceso.
7 Rationale: La autenticación es la primera defensa contra accesos no autorizados.
8 Status: not done
9 Solved by: sgu-reg
10 Priority: users:10

Do you want to overwrite sgu-auth.req?
1 Yes
2 Yes, allow all edits during this session (shift+tab)
3 No

Esc to cancel - Tab to amend
```

rmToo genera automáticamente →



## PDF formal

Generado con LaTeX



## Grafo de trazabilidad

Graphviz automático



## Documentación HTML

Siempre actualizada



## Validación automática

Detecta errores y ciclos

# Conclusión



**rmToo: el enfoque más  
inteligente para requerimientos**

Git • Control total

CI/CD • Siempre actualizado

Open Source • Cero costo

Datos soberanos